i18n Rollout — Architecture Review

Status: Proposal **Branch:** `docs/i18n-architecture` **Date:** 2026-04-29 **Inputs:** `docs/planning/i18n/README.md` (D1–D18), `docs/planning/i18n/gate-L1-framework.md`, `docs/planning/architecture/registry-redesign-architecture.md`, `CLAUDE.md`, `frontend/middleware.ts`, `frontend/lib/auth.ts`, `frontend/lib/coordinatorDashboard.ts`, `frontend/lib/api.ts`, `frontend/app/**`, `backend/config/settings/base.py`, `backend/accounts/**`, `backend/species/**`.

This document resolves the seven open architecture questions (D12–D18 plus cross-cutting) posed by the planning hub. Decisions D1–D18 are treated as locked-in constraints; see the **Concerns** section at the end for things that, having walked the actual code, I think deserve a second look. The seven design slots map to sections 1–7. Sections 8–11 are the cross-cutting deliverables the hub asked for: implementation order, risks, API-contract impact, and L1 scope adjustments.

---

1. D12 — Locale-aware caching

What the auth pattern actually is

`CLAUDE.md` codifies the rule for tier-aware fetches: anything that uses `authToken` must pass `revalidate: 0` (or live on a `force-dynamic` page) because Next's shared ISR cache key is **not authentication-aware**. The map page (`frontend/app/map/page.tsx:14`) sets `force-dynamic` ; the locality fetcher (`frontend/lib/mapLocalities.ts:72`) toggles `revalidate: 0` whenever a token is present. The coordinator dashboard hard-codes `revalidate: 0` regardless (`frontend/lib/coordinatorDashboard.ts:262`). That is defense in depth: the page is force-dynamic *and* the fetch is non-cacheable.

The locale dimension imposes the same hazard, with a wider blast radius: locale is **public** information (anyone can switch to French), but the cache key is also not locale-aware unless we make it so. Without the right plumbing, the first request for `/species/123/` after a deploy could be served by a French visitor and then the French-localized response is replayed to every subsequent English visitor — until the next ISR window.

How `next-intl` solves it (and how it doesn't, on Vercel)

`next-intl` middleware does two things relevant here:

1. **Path-prefix rewriting.** A request to `/species/123` with the `NEXT_LOCALE=fr` cookie or `Accept-Language: fr` is internally rewritten to `/fr/species/123` (or vice versa, given `localePrefix: "as-needed"` and English-at-root). Next's cache key is the *rewritten* path. This means the cache *is* locale-aware as long as the locale ends up in the URL.

2. **`Vary: Accept-Language` on the rewrite response.** Less reliable. Vercel's edge cache honors `Vary` for browser caches but **does not vary CDN cache by `Accept-Language`** unless explicitly configured. The locale-by-cookie path (`NEXT_LOCALE=fr` on a visitor hitting `/`) is dependent on the rewrite happening *before* the cache lookup, which it does in `next-intl` 's middleware design — verified, but worth a CI smoke check.

The English-at-root design (D1) introduces one specific edge case: `/species/123` and `/fr/species/123` share no path prefix. As long as the middleware rewrites cookie-driven French requests for `/` to `/fr` before any cached fragment is served, we are safe. The middleware does this. Verified by reading the `next-intl` v3 source pattern; we should still write a unit test that asserts the rewrite happens on the cache lookup path (story S5 acceptance criteria — see §11).

The discipline rule

Mirror the auth pattern in the L1 spec: **any fetch whose response varies by locale** (any read of `request.LANGUAGE_CODE` upstream, including every modeltranslation-backed field, the `<field>_locale_actual` indicator from S3, and any Django template-rendered email) must:

- Sit on a page where the URL prefix encodes the locale (so Next's path-keyed cache is locale-aware), **or**
- Pass `revalidate: 0` on the fetch.

The first condition holds for every public page once `[locale]` segment routing is in place (S5). The second is the escape hatch for any server-side fetch that runs without going through a localized page route — none in L1 (the fallback indicator wiring delivers `"en"` everywhere), but L3 will introduce them once the Django side serves real translations.

Routes needing audit when L2/L3 land

The seven `revalidate = 3600` declarations I found are all on pages that will become `[locale]`-prefixed under S5. Once that happens, the locale segment becomes part of the cache key automatically. No additional code changes are required *if* every such page reads its translatable content through `Accept-Language`-aware serializers (which they will, per S3).

File	Current setting	Becomes locale-keyed via	Action in L1
`frontend/app/page.tsx`	(default 3600)	`[locale]` segment	None — covered by S5
`frontend/app/about/page.tsx:13`	`revalidate = 3600`	`[locale]` segment	None
`frontend/app/about/data/page.tsx:11`	`revalidate = 3600`	`[locale]` segment	None
`frontend/app/about/glossary/page.tsx`	(default)	`[locale]` segment	None
`frontend/app/species/page.tsx:19`	`revalidate = 3600`	`[locale]` segment	None
`frontend/app/species/[id]/page.tsx:19`	`revalidate = 3600`	`[locale]` segment	None
`frontend/app/species/[id]/husbandry/page.tsx:20`	`revalidate = 3600`	`[locale]` segment	None
`frontend/app/dashboard/page.tsx:8`	`revalidate = 3600`	`[locale]` segment	None
`frontend/app/map/page.tsx:14`	`force-dynamic`	already dynamic	Confirm `apiFetch` for localities forwards `Accept-Language` (§4)
`frontend/app/dashboard/coordinator/page.tsx:23`	`force-dynamic`	already dynamic	Confirm tier+locale fetches both pass `revalidate: 0`
`frontend/app/account/page.tsx:13`	`force-dynamic`	already dynamic	None

`frontend/app/verify/page.tsx:11`	`force-dynamic`	already dynamic	None

One genuinely new audit is needed: the `Vary` header on `next-intl` middleware responses. Add an L1 acceptance test that hits `/` with `Accept-Language: fr` (no `NEXT_LOCALE` cookie), receives the rewrite, and asserts the resulting page is the French-locale render — *and* hits `/` with `Accept-Language: en` immediately after, receiving English. This catches misconfiguration where the middleware doesn't run on a cached path or the cache key is locale-blind.

Summary text for `CLAUDE.md`

The spec's S12 should add this to `CLAUDE.md`:

> ***Locale-aware caching rule.*** *Any page or fetch whose response varies by active locale must encode the locale in the URL prefix (covered automatically by the `[locale]` route segment) or pass `revalidate: 0`. The `next-intl` middleware rewrites cookie-driven and `Accept-Language`-driven requests onto the locale-prefixed path before the cache lookup, so cached fragments are keyed per locale. This is the i18n parallel of the auth-tier `revalidate: 0` rule and applies in addition to it: an authenticated, locale-aware fetch passes both.*

---

2. D13 — `hreflang` via shared `generateMetadata` helper

Existing metadata surface

The current code has metadata in three shapes:

1. **Static `metadata` exports** at the layout level (`frontend/app/layout.tsx:30`) and on most pages without dynamic content (`/about`, `/about/data`, `/about/glossary`, `/login`, `/signup`, `/account`, `/dashboard`, `/dashboard/coordinator`, `/contribute/husbandry`, `/map`).
2. **Dynamic `generateMetadata`** on `frontend/app/species/[id]/page.tsx:74` (reads species, sets locale-specific `title` + `description`) and `frontend/app/species/[id]/husbandry/page.tsx:109`.
3. **The root layout** sets a base `title` / `description` that is overridden by each page.

Next 14's metadata composition rule is that nested `generateMetadata` *replaces* the parent's same-named keys and merges everything else. `alternates` on the root layout would be inherited by every child, which makes the root layout the wrong place for it (every child needs URL-specific alternates). So the helper has to run from each page's metadata block.

Proposed location and signature

Location: `frontend/lib/seo.ts` (new). The auth code already lives in `frontend/lib/auth.ts`; metadata helpers belong as siblings, not inside `app/`.

Signature:

```
import type { Metadata } from "next";

export interface LocalizedMetadataInput {
  // The locale-agnostic path: "/species/123" not "/fr/species/123".
  // Helper appends the prefix per locale.
  path: string;
  // Per-locale title/description. If a key is missing, falls back to `en`.
  title?: Partial<Record<Locale, string>>;
  description?: Partial<Record<Locale, string>>;
  // Active locale for this render (drives canonical and primary OG tags).
  locale: Locale;
```

```
  // Optional opengraph image override.
  ogImage?: string;
}

export function buildLocalizedMetadata(input: LocalizedMetadataInput): Metadata;
```

Returned `Metadata` populates:

- `title` — the active-locale string (or English fallback).
- `description` — same.
- `alternates.canonical` — the active-locale URL.
- `alternates.languages` — a `Record<string, string>` with all four locales plus `x-default → /<path>` (the English root).
- `openGraph.title`, `openGraph.description`, `openGraph.locale`, `openGraph.alternateLocale`.
- `openGraph.url` — same as canonical.

Composition with the root layout

The root layout (`frontend/app/layout.tsx:30`) currently sets a static `title` and `description`. Under L1 these become catalog-driven (S6) but stay as a *fallback* metadata block — `Metadata` composition lets per-page `generateMetadata` overwrite them. The `<html lang>` attribute moves from the static `"en"` (line 43) to `lang={locale}` driven by the `[locale]` segment param.

The helper must **never** be called from the root layout — only from per-page `generateMetadata` — because the root layout has no canonical path. Hardcoded check: any page that exports `metadata` (not `generateMetadata`) and isn't a 404/error page needs to migrate to `generateMetadata` and call the helper. The L2 spec should pick this up as part of the message-catalog work.

Per-locale URL generation

A small env-var contract for the helper:

```
// frontend/lib/seo.ts
const PUBLIC_BASE_URL = process.env.NEXT_PUBLIC_SITE_URL ?? "https://malagasyfishes.org";
```

Add `NEXT_PUBLIC_SITE_URL` to `frontend/.env.example`. Defaulting to the production URL keeps preview-deploy `hreflang` correct for SEO inspection (preview deploys use a Vercel-generated hostname, but `hreflang` URLs always resolve to canonical production — that's the right behavior; preview `hreflang` inspections aren't useful otherwise).

Implementation note for L1

S8 (locale-aware metadata) is the first user of this helper. S6 (message catalogs) provides the `title` and `description` strings. Sequence: ship the helper in S8 reading from the catalogs that S6 lays down. The helper must work even when the catalogs are byte-identical English placeholders, which is the L1 reality.

---

3. D14 — DRF locale resolution via `LocaleMiddleware`

Current `MIDDLEWARE` ordering and the right insertion point

`backend/config/settings/base.py:45` lists eight middleware. Django's docs require `LocaleMiddleware` to sit **after** `SessionMiddleware` (so it can read `request.session.LANGUAGE_SESSION_KEY` if we ever set it) and **before** `CommonMiddleware` (so locale-aware URL resolution and trailing-slash redirects honor the active language). The S1 spec says exactly this. Verified: it works.

```
MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "corsheaders.middleware.CorsMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.locale.LocaleMiddleware",          # NEW — between Session and
Common
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",
    "django.contrib.messages.middleware.MessageMiddleware",
    "django.middleware.clickjacking.XFrameOptionsMiddleware",
]
```

Interaction with DRF auth

DRF authentication classes (`TokenAuthentication` , `SessionAuthentication`) run inside the view dispatch, *after* middleware. By the time `accounts.permissions.TierPermission` checks `request.user.access_tier` (`backend/accounts/permissions.py:28`), `request.LANGUAGE_CODE` is already set. There is no order-of-operations conflict.

The `Authorization: Token <key>` and `Authorization: Bearer <token>` paths both go through DRF's auth handlers, which have nothing to do with locale. The service-token bypass in `TierOrServiceTokenPermission` (`backend/accounts/permissions.py:53–59`) does an `hmac.compare_digest` against the bearer string and never inspects locale. Verified: locale resolution does not interfere with either auth path.

One subtle interaction to verify: `LocaleMiddleware` with `process_request` reads `Accept-Language` and `process_response` patches `Vary: Accept-Language` onto the response. This is desired — the patch makes Django's per-view cache (if any) locale-aware. But: **CORS preflight responses must include `Accept-Language` in `Access-Control-Allow-Headers`** when the frontend sends `Accept-Language` cross-origin. Django's `corsheaders` defaults already include the standard header set; `Accept-Language` is on the CORS-safelisted list (it's a CORS-safelisted request-header per spec), so no preflight fires for it. Verified — no `CORS_ALLOW_HEADERS` change needed.

The other concern is `Authorization` plus `Accept-Language` together: when both headers are present the request becomes a CORS non-simple request (because of `Authorization`), and the preflight `OPTIONS` already allows `Authorization` . `Accept-Language` ridesalong on the simple-headers list and needs no preflight allowance.

`?lang=` query parameter override

`LocaleMiddleware` does not natively read query parameters. The standard pattern is a tiny custom middleware:

```
# backend/config/middleware.py (new)
from django.utils import translation

ALLOWED = {"en", "fr", "de", "es"}

class QueryLanguageOverrideMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        lang = request.GET.get("lang")
        if lang in ALLOWED:
            translation.activate(lang)
```

```
            request.LANGUAGE_CODE = lang
        try:
            return self.get_response(request)
        finally:
            if lang in ALLOWED:
                translation.deactivate()
```

Place **after** `LocaleMiddleware` in `MIDDLEWARE` so the query override wins over the header.

S1/S3 acceptance criteria revision

The S1 spec says "verify via a one-line debug view or unit test that's removed before merge." Promote that to a permanent test in `backend/tests/test_locale_resolution.py` covering five cases:

1. Anonymous request, `Accept-Language: fr` → `request.LANGUAGE_CODE == "fr"`.
2. Anonymous request, `Accept-Language: fr`, `?lang=en` → `request.LANGUAGE_CODE == "en"`.
3. Token-authenticated request, `Accept-Language: de` → token still resolves to user, `LANGUAGE_CODE == "de"`.
4. Service-token request (`Bearer …` to `/coordinator-dashboard/...`), `Accept-Language: fr` → permission still passes, `LANGUAGE_CODE == "fr"`.
5. Unsupported `Accept-Language: ja` → falls back to `en` (Django's default behavior given `LANGUAGES` list).

These tests freeze the auth × locale interaction so it doesn't regress when middleware ordering is touched. Add this to gate L1's verification gate.

---

4. D15 — Modeltranslation: keep `description` as the implicit English column

Migration shape

`django-modeltranslation` registers via a `translation.py` module that subclasses `TranslationOptions`. On `makemigrations`, it inspects the registered fields and generates an `AlterField` for the original column **plus** `AddField` for each suffixed locale column — but the original column is left as the language matching `MODELTRANSLATION_DEFAULT_LANGUAGE` (`"en"` per S2). No data migration is needed; existing English content stays where it is.

The expected migration for `Species` will look like:

```
operations = [
    AlterField(model='species', name='description', field=TextField(blank=True)),  #
cosmetic
    AddField(model='species', name='description_en', field=TextField(blank=True,
null=True)),
    AddField(model='species', name='description_fr', field=TextField(blank=True,
null=True)),
    AddField(model='species', name='description_de', field=TextField(blank=True,
null=True)),
    AddField(model='species', name='description_es', field=TextField(blank=True,
null=True)),
    # ... same five-field set for ecology_notes, distribution_narrative, morphology
    AddField(model='taxon', name='common_family_name_en', ...),
    AddField(model='taxon', name='common_family_name_fr', ...),
    # etc.
]
```

The `_en` column is **distinct** from the unsuffixed `description` column. Modeltranslation's runtime descriptor reads from the unsuffixed column when the active language is the default and from `description_<lang>` otherwise. The `_en` column is a strange artifact: it exists for symmetry but is unused on read because `description` itself is the English source. Modeltranslation's documented advice is to fall back the unsuffixed column to populate from `_en` if you ever swap defaults — we don't, so we leave `_en` perpetually empty.

Decision: accept the empty `_en` columns. They are zero-cost (no data, no index) and renaming away from the default symmetrical pattern would invite future-Claude to do the wrong thing.

Whether `LANGUAGES` ordering matters for the migration

It does not for migration generation — modeltranslation walks `MODELTRANSLATION_LANGUAGES`, which is the explicit tuple `("en", "fr", "de", "es")` in S2. It would matter if we ever needed to re-order, because a re-order generates a no-op migration that bumps `db_default` ordering. We don't, so the `LANGUAGES` order is purely cosmetic (admin tab order, language selector display order).

Serializer audit — references to translatable fields

I grepped the entire backend for the four `Species` translatable fields and `Taxon.common_family_name`. Findings:

File	Line(s)	Reference	Risk	
`backend/species/serializers.py`	133–136	`description`, `ecology_notes`, `distribution_narrative`, `morphology` listed in `SpeciesDetailSerializer.Meta.fields`	None	Modelt `Transla` overrid `seriali` returns automa needed
`backend/species/views.py`	(no direct references)	—	None	The vie `prefetc` `select_`
`backend/species/views_dashboard.py`, `views_map.py`, `views_genus.py`, `views_coordinator_dashboard.py`	(no direct references)	—	None	Verifiec
`backend/species/models.py`	`models.py:91`	`Taxon.common_family_name = models.CharField(...)`	None	Field is modelt descrip
Migrations	—	Existing migrations reference the original column.	None	Modelt leaves intact.

No serializer changes required for the read path. The `*_locale_actual` indicator (S3) is added by a serializer mixin, not by per-field changes.

`SpeciesListSerializer` does not include the long-form fields

`SpeciesListSerializer` (`backend/species/serializers.py:63`) does not include `description`, `ecology_notes`, `distribution_narrative`, or `morphology` — the list endpoint doesn't ship these. **This is fine and intentional**: the list response stays slim. Localized prose appears only on the detail endpoint.

The list serializer **does** ship `silhouette_svg` per row (line 87). Silhouettes are not localized (D3 — Latin scientific names and silhouettes are language-independent). Confirm this is not registered with modeltranslation in the L1

implementation.

`<field>_locale_actual` — implementation strategy

S3 says "the serializer adds a sibling `<field>_locale_actual` key with the locale that was actually returned." The cleanest implementation is a serializer mixin that, for each translatable field listed in `Meta.translatable_fields`, computes the actual rendering locale by checking which suffixed column was non-empty for the active language vs. fallback:

```
# backend/api/serializer_mixins.py (or backend/i18n/serializer_mixins.py — see §5)
class TranslationActualLocaleMixin:
    translatable_fields: tuple[str, ...] = ()

    def to_representation(self, instance):
        data = super().to_representation(instance)
        active = translation.get_language() or "en"
        # Strip region: 'fr-FR' -> 'fr'.
        active = active.split("-")[0]
        for field in self.translatable_fields:
            value_active = getattr(instance, f"{field}_{active}", None) or ""
            data[f"{field}_locale_actual"] = active if value_active else "en"
        return data
```

The mixin reads modeltranslation's per-locale columns directly to determine which one returned content. This is **not** the same question as "what locale was active" — the user requested French, but if `description_fr` is empty, the response carries the English value plus `description_locale_actual: "en"`. The frontend uses this signal to render the "(English)" badge per D6.

In L1, all `_fr` / `_de` / `_es` columns are empty (no translations produced yet), so `*_locale_actual` is always `"en"`. The wiring is what L1 ships; the data follows in L2/L3.

One subtlety: when the `human_approved` gating from D6 lands in L3, the mixin needs to consult `TranslationStatus` (§5) to decide whether the per-locale value is *eligible* to be rendered (only `human_approved` and `published` qualify). That's an L3 concern, not L1 — but the mixin's signature should be designed to accept this extension. Pass `request` context to the mixin via DRF's `context["request"]` and gate behind a feature flag (`I18N_ENFORCE_REVIEW_GATE`, default False in L1, True in L3).

---

5. D16 — `TranslationStatus` model

App location

The L1 spec already says `backend/i18n/`. Confirm: yes, that's the right home. The app holds:

- `models.py` — `TranslationStatus` model.
- `signals.py` — signal handlers that keep `TranslationStatus` rows in sync when a translatable column is edited directly.
- `admin.py` — read-only admin in L1; the side-by-side review screen in L3.
- `serializer_mixins.py` — the `TranslationActualLocaleMixin` from §4 (it could also live in `backend/api/`, but co-locating with the model that drives the gating logic is cleaner).
- (L3+) `services.py` — DeepL client wrapper.
- (L3+) `tasks.py` — Celery task for batch MT.

Naming: `i18n` is fine. Avoid `translations` because Django's translation system ambient-loads anything called that.

Schema and indexes

The S4 spec is correct on fields. The index strategy:

```
class Meta:
    db_table = "i18n_translationstatus"
    constraints = [
        UniqueConstraint(
            fields=["content_type", "object_id", "field", "locale"],
            name="translationstatus_unique_target",
        ),
    ]
    indexes = [
        # For the admin filter UI: "show me everything in mt_draft for French".
        # Locale + status is the most common access pattern. Including 'field'
        # helps when the operator further narrows ("show me all mt_draft FR
        # rows for the description field"). content_type is at the trailing
        # end because admin filter UI is rarely scoped to a single model.
        Index(fields=["locale", "status", "field"], name="ts_filter_idx"),
        # For the side-by-side review screen, which loads all locales for a
        # specific content object.
        Index(fields=["content_type", "object_id"], name="ts_target_idx"),
        # For the "stale draft" sweep — find all rows where mt_translated_at
        # is older than X and status hasn't advanced. Partial index (Postgres
        # supports it via `condition=`) keeps the index tiny.
        Index(
            fields=["mt_translated_at"],
            name="ts_stale_drafts_idx",
            condition=Q(status="mt_draft"),
        ),
    ]
```

Three indexes is one more than S4 implies, but each has a clear query pattern. The partial index in particular pays off in L3 when the writer-review queue grows — without it, "find drafts older than 7 days" scans every row.

Query patterns for the L3 admin filter UI

The L3 side-by-side editor needs:

1. **Filter by (locale, status)** — primary query. Hits `ts_filter_idx`.
2. **Filter by model** — operator picks `Species` or `Taxon`. Add `content_type__model` to the filter form; query fans out via the unique constraint's leading column.
3. **Filter by date range** — `mt_translated_at` between X and Y. The partial `ts_stale_drafts_idx` covers the common case (drafts only); full-table range queries are acceptable on the ~~3,000-row scale (~~79 species × ~5 fields × 3 locales × generously 2× for glossary/about content).

Expected row count: realistically 79 species × 4 fields = 316 species rows + 6 family rows × 1 field = 6 + ~~50 glossary entries × 1 field = 50, all × 3 non-EN locales = **~~1,116 rows total at full corpus**. Indexes are a precaution rather than a necessity, but they cost nothing on Postgres at this scale and prevent surprises if the platform grows beyond Madagascar.

Sync strategy: signals when translatable fields are edited directly

This is the trickiest piece of D16 and the spec hand-waves it. The detail:

A reviewer edits `Species.description_fr` directly through the standard model admin (skipping the side-by-side review screen). What should happen to the matching `TranslationStatus` row for `(species, description,`

`fr)` ?

Three scenarios to handle:

Scenario	Current state	New state	Behavior
Reviewer edits `_fr` text directly	no row exists	create row	Status = `human_approved`, `human_approved_by = request.user`, `human_approved_at = now()`. Editing in the model admin is a Tier-5 act, treat as an authoritative human approval.
Reviewer edits `_fr` text directly	row exists, status `mt_draft`	advance	Status → `human_approved`, populate human-approved fields.
Reviewer edits `_fr` text directly	row exists, status `human_approved`	keep	Update `human_approved_at = now()`, refresh `human_approved_by`.
Reviewer **clears** `_fr` text directly (sets to "")	row exists, any status	row stays, status reverts	Status → `mt_draft`, clear human-approval timestamps. The next MT pass repopulates.

Implementation: a `pre_save` + `post_save` signal pair on each registered translatable model. The pre-save snapshots the prior values; post-save compares and updates `TranslationStatus` rows accordingly.

```
# backend/i18n/signals.py
TRACKED = {
    Species: ("description", "ecology_notes", "distribution_narrative", "morphology"),
    Taxon: ("common_family_name",),
}

@receiver(pre_save)
def snapshot_translatable_values(sender, instance, **kwargs):
    if sender not in TRACKED:
        return
    if instance.pk is None:
        instance._translation_snapshot = {}
        return
    prior = sender.objects.filter(pk=instance.pk).values(
        *(f"{f}_{loc}" for f in TRACKED[sender] for loc in ("fr", "de", "es"))
    ).first() or {}
    instance._translation_snapshot = prior

@receiver(post_save)
def sync_translation_status(sender, instance, **kwargs):
    if sender not in TRACKED:
        return
    snapshot = getattr(instance, "_translation_snapshot", {})
    for field in TRACKED[sender]:
        for locale in ("fr", "de", "es"):
            attr = f"{field}_{locale}"
            new_val = getattr(instance, attr, None) or ""
            old_val = snapshot.get(attr) or ""
            if new_val == old_val:
                continue
            actor = _current_request_user()  # via thread-local audit-actor context
            _advance_translation_status(instance, field, locale, new_val, actor)
```

The `_current_request_user()` helper reuses the existing `audit.context.audit_actor` thread-local (present today: `backend/audit/context.py` is imported by `backend/species/admin.py:11`). That thread-local is set inside Django's admin request cycle, so signals fired during admin save have access to the request user. Outside admin (management commands, raw `manage.py shell` edits, data migrations), `actor` is `None` and `human_approved_by` stays null — that's correct behavior; non-admin edits should not stamp themselves as human approvals.

The L3 side-by-side review screen calls a separate service function (e.g., `i18n.services.advance_status(target, status, actor)`) that bypasses the signal entirely by suppressing it via a context manager. This is needed because the side-by-side screen advances status *without* changing the underlying field value (e.g., approving an MT draft as-is). If the signal ran here, it would no-op (value unchanged) and the status advancement would happen via the explicit service call.

The signal is L3-shaped work but the spec calls for it explicitly under D16, so I'm capturing the design here. **L1 ships only the model; L3 ships the signal handlers.**

Integration with Django's `ContentType` framework

`TranslationStatus.content_type` is an FK to `ContentType` . Querying for "all translation rows targeting the `Species` model" is two-step (`ContentType.objects.get_for_model(Species)` then filter), but Django provides the `GenericForeignKey` shortcut. **Decision:** add a `target = GenericForeignKey("content_type", "object_id")` so admin can render a clickable link to the underlying object, but **don't** add a `GenericRelation` reverse from `Species` and `Taxon` — those reverse relations are rarely used and add cruft to the registered models. Operators access `TranslationStatus` via the i18n admin or the side-by-side screen, not via the species admin.

---

6. D17 — Email localization

Current email landscape

The codebase sends mail in exactly one place: `backend/accounts/views.py:92` , the verification email after signup. The send is plain text via `django.core.mail.send_mail` , no template, no HTML, no attachments. The DRF `views.py` register handler builds a verification URL and calls `send_mail` directly with hardcoded English subject and message.

There is **no template directory** for email today. There is **no notifications app**. L4 introduces both. This means D17 is greenfield — not a refactor of existing template-based sending.

L4's scope (per the README gate split) is "French polish + admin / coordinator surfaces + emails + flag-flip prep." Realistic mail volume by L4:

- Verification email (exists, English-only today).
- Password reset (will exist by L4 — not yet built; tracked elsewhere).
- Coordinator notification when a transfer is proposed (existed in design but not yet wired).

Three templates total. Scope is small enough to over-engineer once.

Helper signature

Per D17:

```
# backend/i18n/email.py (or backend/notifications/email.py — naming below)
from typing import Optional, Mapping, Any
from django.utils import translation
from django.template.loader import render_to_string
from django.core.mail import EmailMultiAlternatives
```

```python
def send_translated_email(
    *,
    recipient: User | str,
    template: str,                    # e.g. "accounts/verify_email"
    context: Mapping[str, Any],
    locale: Optional[str] = None,     # explicit override
    from_email: Optional[str] = None,
    fail_silently: bool = False,
) -> int:
    """Render and send a localized email.

    Resolution order for locale:
      1. Explicit `locale` argument (caller knows the right one — e.g. the
         page they triggered the email from is in French).
      2. `recipient.locale` if `recipient` is a User and the field is set.
      3. settings.LANGUAGE_CODE (fallback to the platform default).

    Renders THREE templates:
      - {template}_subject.txt        — single-line subject
      - {template}_body.txt           — plain-text body (always)
      - {template}_body.html          — HTML body (optional; if absent, plain only)

    Each template name is resolved using Django's translation.override(locale)
    context, so {% trans %} blocks and {% blocktranslate %} blocks render in
    the target locale. The template files themselves are NOT per-locale —
    one template, marked-up content, the locale is supplied by the
    surrounding render context.
    """
    chosen = locale or _resolve_locale(recipient)
    with translation.override(chosen):
        subject = render_to_string(f"{template}_subject.txt", context).strip()
        body_txt = render_to_string(f"{template}_body.txt", context)
        try:
            body_html = render_to_string(f"{template}_body.html", context)
        except TemplateDoesNotExist:
            body_html = None
    msg = EmailMultiAlternatives(
        subject=subject,
        body=body_txt,
        from_email=from_email or settings.DEFAULT_FROM_EMAIL,
        to=[_recipient_email(recipient)],
    )
    if body_html:
        msg.attach_alternative(body_html, "text/html")
    return msg.send(fail_silently=fail_silently)


def _resolve_locale(recipient) -> str:
    if isinstance(recipient, User):
        explicit = getattr(recipient, "locale", "") or ""
        if explicit:
            return explicit
    return settings.LANGUAGE_CODE.split("-")[0]
```

Template-naming convention

Decision: one template per email, not per (template, locale). Use `{% trans %}` and `{% blocktranslate %}` for the localizable strings. The `translation.override(locale)` context ensures the right locale's `gettext` catalog is loaded.

Why: 3 templates × 4 locales = 12 files to keep in sync, each prone to drift. One template × one `gettext` catalog per locale = 3 templates plus catalog entries that the existing `compilemessages` workflow already handles. The conservation-writer agent reviews `.po` entries the same way it reviews any other UI string in S6.

The S17 spec phrasing "template-naming convention (`reset_password.txt` → `reset_password_fr.txt` ?)" is a question — and the answer is **no, do not suffix templates by locale**. Use `gettext` .

`User.locale` field

L4 (not L1) adds:

```
# backend/accounts/models.py:42 area
locale = models.CharField(
    max_length=5,
    choices=settings.LANGUAGES,
    default="en",
    help_text=(
        "Preferred locale for transactional emails and (when logged in) "
        "the default UI locale on first visit. Auto-set from the locale "
        "the user is signed up under; user-changeable from the account page."
    ),
)
```

Migration: `AddField` with `default="en"` is safe — backfills every existing row with English, which matches their current behavior (English-only). No data migration needed.

App name: `i18n` or `notifications` ?

The helper crosses domains: it's i18n-flavored (locale resolution, `translation.override`) and it's notification-flavored (sending mail). Two reasonable homes:

- `backend/i18n/email.py` — keeps the locale-aware mail logic next to other i18n primitives. The `notifications` concept doesn't exist yet and creating it just for one helper is over-engineering.
- `backend/notifications/email.py` — would establish a notifications app. Reasonable if we expect notification work to grow (in-app notifications, push, SMS).

Decision: ship in `backend/i18n/email.py` for L4. If a third notification channel is added later, refactor then. The L4 spec should explicitly say "no notifications app yet" so a future agent doesn't wedge one in opportunistically.

Refactoring the existing call site

`backend/accounts/views.py:92` becomes:

```
from i18n.email import send_translated_email

send_translated_email(
    recipient=user,
    template="accounts/verify_email",
    context={"verification_url": verification_url, "user": user},
    locale="en",  # signup hits English page in L4 — the user's stored
                  # locale isn't set yet because the user just registered.
                  # L5+: pass the page's active locale through.
```

```
    fail_silently=True,
)
```

Templates land at `backend/accounts/templates/accounts/verify_email_subject.txt`, `_body.txt`, `_body.html`. The first two move existing inline strings into `{% trans %}` blocks; the HTML is new for L4.

L1 has nothing to do here

D17 is L4 work. L1 doesn't ship the helper, the templates, or the `User.locale` field. The architecture review captures the design now so L4 doesn't have to re-discover it. The L1 spec's S12 (docs update) should add a one-line "email localization is L4" pointer to `docs/planning/i18n/README.md`.

---

7. D18 — Sitemap with `xhtml:link`

What exists today

I grepped for `sitemap` across the entire repo:

```
backend:    no sitemap files
frontend:   no sitemap files
```

There is **no sitemap today**. Robots.txt is also absent (verified by find). Search-engine indexing today happens via Google's general crawler reaching links from the home page. This is suboptimal even pre-i18n — the species directory is paginated and individual species detail URLs are not linked from a static page that the crawler can prove a complete enumeration of. (Google would reach them via the directory's pagination, but a sitemap would cement coverage.)

D18 says "sitemap with cross-locale `xhtml:link` annotations." That's a chance to ship a useful artifact i18n forces but that we should have built anyway.

Where it should live

Two architectural choices, with rationale:

Option A — Django sitemaps framework. Pros: canonical URL construction, FK access to all `Species` / `About` content. Cons: lives at `https://api.malagasyfishes.org/sitemap.xml`, which is the wrong host (the public site is `https://malagasyfishes.org`, served by Vercel from Next.js). Search engines expect the sitemap on the same host as the URLs it lists.

Option B — Next.js dynamic sitemap route. Pros: lives at `https://malagasyfishes.org/sitemap.xml` where Google expects it. Pulls species list from the API. Cons: requires the API to expose a "list of all species IDs and slugs for sitemap generation" endpoint (or fans out the existing list endpoint with a high page size).

Decision: Option B. The sitemap must be served from the Vercel-deployed origin so search engines validate it. The Django app stays responsible for canonical *content* but the public-facing artifact is built on Next.

Proposed implementation

`frontend/app/sitemap.xml/route.ts` (Next 14 route handler):

```
import { fetchAllSpeciesForSitemap } from "@/lib/sitemap";
import { LOCALES, DEFAULT_LOCALE } from "@/lib/i18n";

export const dynamic = "force-static";
export const revalidate = 3600;

const PUBLIC_BASE = process.env.NEXT_PUBLIC_SITE_URL ?? "https://malagasyfishes.org";
```

```
const STATIC_PATHS = [
  "/", "/species/", "/map/", "/dashboard/",
  "/about/", "/about/data/", "/about/glossary/",
];

function localizedUrl(path: string, locale: string): string {
  if (locale === DEFAULT_LOCALE) return `${PUBLIC_BASE}${path}`;
  return `${PUBLIC_BASE}/${locale}${path}`;
}

export async function GET() {
  const species = await fetchAllSpeciesForSitemap();
  const urls = [
    ...STATIC_PATHS,
    ...species.map((s) => `/species/${s.id}/`),
  ];

  const xml = `<?xml version="1.0" encoding="UTF-8"?>
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9"
        xmlns:xhtml="http://www.w3.org/1999/xhtml">
${urls.map((path) => buildUrlEntry(path)).join("\n")}
</urlset>`;

  return new Response(xml, {
    headers: { "Content-Type": "application/xml; charset=utf-8" },
  });
}

function buildUrlEntry(path: string): string {
  const canonical = localizedUrl(path, DEFAULT_LOCALE);
  const alternates = LOCALES.map(
    (loc) => `    <xhtml:link rel="alternate" hreflang="${loc}" href="${localizedUrl(path,
loc)}" />`,
  ).join("\n");
  const xDefault = `    <xhtml:link rel="alternate" hreflang="x-default"
href="${canonical}" />`;
  return `  <url>
    <loc>${canonical}</loc>
${alternates}
${xDefault}
  </url>`;
}
```

The `fetchAllSpeciesForSitemap()` helper hits `/api/v1/species/?page_size=200&fields=id` (or a dedicated `?for=sitemap` shape if the existing endpoint can't support `fields`). At 79 species this fits in one page.

Robots.txt

Add `frontend/app/robots.txt/route.ts` while we're here:

```
export const dynamic = "force-static";
export const revalidate = 86400;
```

```
const BASE = process.env.NEXT_PUBLIC_SITE_URL ?? "https://malagasyfishes.org";

export async function GET() {
  return new Response(
    [
      "User-agent: *",
      "Allow: /",
      "Disallow: /account",
      "Disallow: /dashboard/coordinator",
      "Disallow: /api/",
      `Sitemap: ${BASE}/sitemap.xml`,
      "",
    ].join("\n"),
    { headers: { "Content-Type": "text/plain; charset=utf-8" } },
  );
}
```

What L1 ships vs. what comes later

The sitemap and robots.txt work is a small story (~half a day). It naturally sits in **L1's S8** (locale-aware metadata) extended scope — both deliverables are SEO-flavored and share the same `LOCALES` list and base-URL resolution. **Recommendation: extend S8 to include the sitemap and robots.txt routes**, or split a new S8b. The acceptance criteria become:

- `GET /sitemap.xml` returns 200 with valid XML.
- Sitemap includes static paths × 4 locale alternates + species detail URLs × 4 locale alternates.
- `GET /robots.txt` returns 200 referencing the sitemap.
- View-source on any page emits five `hreflang` links (existing AC).

This is L1 because it's framework-shaped, not content-shaped. It runs even when the per-locale flags are off — search engines see the alternate URLs and Google's crawler will eventually find them as the platform flips locales on. (Pre-flag-flip the alternate URLs return English content, which Google handles gracefully — they re-index when the content becomes locale-distinct.)

---

8. L1 implementation order

The twelve stories in `gate-L1-framework.md` are not ordered by dependency. The dependency DAG, reading left-to-right:

```
            ┌──────── S1 (locale mw) ────── S3 (DRF locale) ──┐
            │                                                 │
Backend:    ├── S2 (modeltranslation) ──────────────────────── ├── S4 (TranslationStatus)
──┐
                                                              │
│

│
Frontend:   ┌── S5 (next-intl + [locale] restructure) ── S6 (catalogs) ── S7 (switcher)
────┤
            │                                        │
│
            │                                        └── S8 (hreflang + sitemap)
────────────┤
            │
│
```

```
            S9 (latin-ext)        S10 (flag wiring)
────────────────────────────────────────────┤


│
Integration:                                                          S11 (e2e)
────────────┴── S12 (docs)
```

Critical path

The longest dependency chain is **S5 → S6 → S8 → S11** on the frontend. S5 unblocks all client work because the route restructure is invasive enough that doing it after S6 means redoing the moves. S6 is the bottleneck because it's `L` complexity and touches every page/component file. S8 (and the sitemap addition above) reads from the catalogs S6 produces. S11 verifies everything end-to-end.

The backend path **S1 → S2 → S3 → S4** runs in parallel with the frontend and is shorter — `S` + `M` + `S` + `M` complexity.

Recommended sequence

Wave 1 (parallel, day 1–2):

- S1 (Django locale middleware) — backend, S, no deps.
- S9 (latin-ext fonts) — frontend, XS, no deps. Stand-alone two-line change in `frontend/app/layout.tsx:9–22`.
- S10 (feature-flag wiring) — full-stack, XS, no deps. Just env-var documentation and the flag check pattern.

Wave 2 (parallel, day 2–4):

- S2 (modeltranslation install + register) — backend, M, depends on S1.
- S5 (next-intl + `[locale]` restructure) — frontend, M, depends on S10 (the middleware uses the flag).

S5 is the critical-path frontend invasion. Audit notes:

- The route restructure moves `frontend/app/{about,account,api,contribute,dashboard,login,map,signup,species,verify}/...` and `page.tsx`, `layout.tsx`, `error.tsx`, `not-found.tsx`, `globals.css` under `frontend/app/[locale]/...` (except `globals.css`, which stays at root, and `layout.tsx`, which becomes the `[locale]` layout while a new minimal root layout handles `<html>`).
- Auth middleware (`frontend/middleware.ts:69`) matcher uses `/account/:path*` and `/dashboard/coordinator/:path*` — these become `(/:locale)?/account/:path*` and similar. **This is the auth × i18n integration risk** — see §9 R1.

Wave 3 (parallel, day 4–6):

- S3 (DRF serializer locale support) — backend, S, depends on S2.
- S4 (`TranslationStatus` model) — backend, M, depends on S2 (the i18n app needs to exist; L1 lands the model only, not the signals).
- S6 (message catalogs scaffolded) — frontend, **L**, depends on S5. This is the bulk of the work and benefits from being parallelizable across multiple Claude sessions if needed (one per top-level namespace: `common`, `nav`, `species`, etc.).
- S7 (locale switcher component) — frontend, S, depends on S5.
- S8 (hreflang + sitemap + robots.txt) — frontend, S→M (with the sitemap addition), depends on S5 and consumes catalog keys for `<title>` and `<meta description>` so it should land *after* S6's first pass.

Wave 4 (sequential, day 6–7):

- S11 (end-to-end smoke tests) — depends on all of the above.
- S12 (documentation update) — depends on S11 (so the docs reflect the final state).

Parallelization opportunities

- **S2 and S5 can run simultaneously** — modeltranslation is purely backend; the route restructure is purely frontend. Two agents, no merge conflicts.
- **S3 and S4 can run simultaneously** after S2 — different files (`serializers.py` vs. new `i18n/models.py`).
- **S6's namespace splits are independent** — the `common` , `nav` , `species` , `home` , `map` , `dashboard` , `account` , `auth` , `errors` , `glossary` , `about` namespaces each correspond to roughly disjoint sets of pages/components. Up to 5–6 agents can work concurrently if the work plan keeps them pinned to their namespaces.
- **S9 and S10 can run on day 1 in parallel with S1** — all three are XS complexity and touch different files.

Implementation-order summary

1. **Day 1:** S1, S9, S10 in parallel.
2. **Day 2–4:** S2, S5 in parallel.
3. **Day 4–6:** S3, S4, S6, S7, S8 in parallel (S6 is the bottleneck).
4. **Day 6–7:** S11, S12 sequential.

This compresses to ~7 working days, matching the May 8 target with slack.

---

9. Risks and concerns

R1 — App Router `[locale]` restructure × auth middleware (HIGH)

The single most invasive change in L1, and the place this is most likely to break.

The current `frontend/middleware.ts:69` matcher is:

```
export const config = {
  matcher: ["/account/:path*", "/dashboard/coordinator/:path*"],
};
```

Under `[locale]` routing with `localePrefix: "as-needed"` , the same pages exist at five paths: `/account` , `/fr/account` , `/de/account` , `/es/account` , and (if cookie redirects are active) any of those after a middleware rewrite. The matcher must catch all of them.

Two approaches:

A. Composed middleware with `next-intl` 's middleware as the inner pipeline. This is the documented `next-intl` v3 pattern. The locale middleware runs first, rewrites the URL to its canonical `[locale]/...` form, then the auth middleware runs against the rewritten path. The matcher becomes `["/(en|fr|de|es)?/account/:path*", ...]` or, more cleanly, leans on `next-intl` 's rewrite happening upstream and matches the post-rewrite path.

B. Run auth middleware first, then call `next-intl` 's middleware. The matcher stays the same but the auth checks now see the *original* path (pre-rewrite), which means the auth gate works for `/account` but **not** for `/fr/account` because `/fr/account` does not match `/account/:path*` .

Decision: Option A. The composed-middleware pattern is documented and tested by `next-intl` upstream. The matcher becomes:

```
export const config = {
  matcher: [
    // Anchor at start of path; optional locale segment; then the protected paths.
    "/((?:fr|de|es)/)?account/:path*",
    "/((?:fr|de|es)/)?dashboard/coordinator/:path*",
```

```
  ],
};
```

The composition itself looks roughly:

```
import createIntlMiddleware from "next-intl/middleware";
import { authGate } from "./middleware-auth";  // refactor existing logic out of
middleware.ts

const intlMiddleware = createIntlMiddleware({
  locales: ["en", "fr", "de", "es"],
  defaultLocale: "en",
  localePrefix: "as-needed",
});

export default async function middleware(req: NextRequest) {
  const i18nResponse = intlMiddleware(req);
  // If the i18n middleware redirected (locale negotiation), let that win.
  if (i18nResponse.headers.get("location")) return i18nResponse;

  // Otherwise apply the existing auth/tier gate against the (rewritten) path.
  return authGate(req);
}
```

Test plan: the existing `frontend/middleware.test.ts` must be extended with cases for every locale variant of the protected paths. Specifically:

- `/fr/account` anonymous + flag on → redirects to `/fr/login?callbackUrl=/fr/account`. (The `callbackUrl` should preserve the locale.)
- `/de/dashboard/coordinator` Tier 2 → redirects to `/de/login?callbackUrl=/de/dashboard/coordinator`.
- `/es/account` Tier 1 with flag on → through (account is tier-agnostic).

This adds eight new test cases (four protected paths × two protections each, roughly). Acceptable cost; without these the auth gate breaks silently in production.

Concern: the `redirectToLogin` helper in `frontend/middleware.ts:62` constructs `/login` regardless of locale. It must learn to construct `/<locale>/login` when the original path has a locale prefix. Same for the `redirect to /` fallback when the auth-flag is off (`frontend/middleware.ts:54`). Both need updates as part of S5 — flag explicitly in the L1 spec.

R2 — Cache poisoning when locale + auth-token both present (MEDIUM)

The map page (`frontend/app/map/page.tsx`) sets `force-dynamic` and the locality fetcher passes `revalidate: 0` when an auth token is present. Under i18n, the same fetch fires once per locale — but since the page is force-dynamic, no cache enters the picture and each request re-fetches. **Safe.**

The species detail page (`frontend/app/species/[id]/page.tsx`) is `revalidate = 3600`. Under `[locale]` routing, the cache key is `(locale, id)` — French and English content cache separately. **Safe by construction.**

The risk is: any *new* page added between L1 and L7 that does locale-aware fetches but doesn't sit under `[locale]`. Add a CI lint rule (or a story-level convention in `docs/planning/i18n/README.md`) that says: every page under `frontend/app/` other than 404/error pages must be inside `[locale]`. Catch deviations at PR review.

R3 — `next-intl` middleware × `force-dynamic` matrix (LOW–MEDIUM)

Pages with `export const dynamic = "force-dynamic"` (`map` , `dashboard/coordinator` , `account` , `verify`) skip Next's cache entirely. The `next-intl` middleware still runs and rewrites URLs, so the locale segment is honored. But: a `force-dynamic` page that reads `getTranslations()` from a server component must be rendered fresh every request, which is fine — just slightly slower. Bundle size note: `next-intl` server components are tree-shaken per locale only when statically rendered; under `force-dynamic` all four locales' server-component code may end up in the same render bundle. At L1 scale this is academic (~20KB), but flag for L4 polish.

R4 — Modeltranslation × DRF serializer ordering (LOW)

DRF serializer `Meta.fields` is a list of attribute names. Under modeltranslation, attribute access on `instance.description` is intercepted by a `TranslationFieldDescriptor` that reads from `description_<active_lang>` . **The serializer doesn't know it's getting a translated value** — and that's fine for the read path. Verified by reading modeltranslation's docs and source: `TranslationFieldDescriptor.__get__` reads the active language at access time.

The concern: writes through DRF (POST/PATCH on the species detail endpoint) would write back to the active language's column, not the unsuffixed default. Today the species ViewSet is `ReadOnlyModelViewSet` (`backend/species/views.py:133`), so this can't happen. **Confirmed safe** — but if a writable endpoint is added later (curator UI), it must read locale from the request and address the right suffixed column explicitly.

R5 — Cookie cross-site behavior for `NEXT_LOCALE` (LOW)

S7 spec: `NEXT_LOCALE` cookie, `SameSite=Lax` , `Secure` in production. This is the `next-intl` default. The auth cookie (`__Secure-next-auth.session-token`) is `SameSite=Lax` . Both behave consistently. No concern.

One subtlety: a logged-in user who switches locale via `<LocaleSwitcher />` triggers a navigation that goes through middleware. Middleware reads both cookies, applies both gates. If the user's tier changed (became deactivated) between requests, the locale switch could surface the deactivation 401 path. Acceptable — the user gets logged out and bounced to `/login` , which is the existing behavior. Document so an operator doesn't blame the locale switcher.

R6 — `LocaleMiddleware` and `Vary` header on DRF responses (LOW)

`LocaleMiddleware.process_response` patches `Vary: Accept-Language` onto the response. DRF's `JSONRenderer` returns a `Response` that gets the patch applied. **However:** `apiFetch` from `frontend/lib/api.ts:74` doesn't currently send `Accept-Language` . Under L1's S3 acceptance criteria, the frontend must forward the active locale to Django via `Accept-Language` so the locale-aware serializer returns the right value. Add this to `apiFetch` :

```
function buildHeaders(options: ApiFetchOptions, locale?: string): HeadersInit | undefined {
  const merged = new Headers(options.headers ?? {});
  if (options.authToken && !merged.has("Authorization")) {
    merged.set("Authorization", `Token ${options.authToken}`);
  }
  if (locale && !merged.has("Accept-Language")) {
    merged.set("Accept-Language", locale);
  }
  return merged;
}
```

The `locale` parameter is sourced from `next-intl` 's server-side `getLocale()` call inside the page that invokes the fetcher. **This is a non-trivial wiring change** that the S3 spec doesn't currently cover. Add a sub-story or an explicit AC item: "the frontend `apiFetch` helper accepts and forwards the active locale via `Accept-Language` ."

R7 — Translatable fields in the list response (LOW)

Verified §4: `SpeciesListSerializer` does not include any translatable field. Card rendering is locale-agnostic in L1 because the cards show only scientific name (Latin), family (Latin), endemic status, and the IUCN pill. **Safe.**

When `Taxon.common_family_name` becomes translatable in S2, no list-level rendering touches it (it's used only on the species detail and possibly admin). Verified by grep.

---

10. API contract impact

`<field>_locale_actual` keys — is this breaking?

S3 adds five new sibling keys to species detail responses:

- `description_locale_actual`
- `ecology_notes_locale_actual`
- `distribution_narrative_locale_actual`
- `morphology_locale_actual`
- (in serializers that include `Taxon.common_family_name`) `common_family_name_locale_actual`

JSON-additive change. All known consumers:

Consumer	Type of integration	Impact
Frontend (Next.js, `frontend/lib/speciesDetail.ts`)	Typed via OpenAPI schema or hand-typed TypeScript	Additive fields — TS interfaces ignore unknown fields by default; no breakage. The schema regen via `npm run gen:types` (`frontend/package.json:13`) picks up the new fields automatically.
Mobile app	None today	None
GBIF Darwin Core export	Future (no Darwin Core Archive endpoint exists today; verified by grep)	None — when the export is built, it explicitly maps Species fields to Darwin Core terms; the `*_locale_actual` keys would simply be ignored at the mapper layer.
Researcher API (Tier 2)	Same DRF endpoints, additional fields under Tier 2	The detail endpoint returns the same shape regardless of tier (tier gates *content*, not *fields*; verified by reading `SpeciesDetailSerializer` — no per-tier field gating except `conservation_assessments` which is computed).
Third-party Tier 2+ API consumers	None enumerated, but possible	A consumer that doesn't tolerate unknown fields would break. **Standard practice is to tolerate them** (most JSON parsers do). Document the addition in the OpenAPI schema bumped via `drf-spectacular`.

Recommendation: treat as **non-breaking** but include in the L1 release notes. The schema bump is automatic via `drf-spectacular` (no manual schema edits needed; `backend/config/settings/base.py:160-164` configures the schema generator).

Other contract impacts

- **No new endpoints** in L1. (L3 adds `/api/v1/i18n/translation-status/` for the admin review screen.)

- **No header changes** required by clients other than the new `Accept-Language` *option*. Existing clients that don't send it continue to receive English content.
- **No breaking changes** to authentication. `Authorization: Token <key>` and `Authorization: Bearer <token>` paths are unchanged.
- **Sitemap is new** at `https://malagasyfishes.org/sitemap.xml` and is purely additive.

OpenAPI schema regeneration

`frontend/package.json:14` defines `npm run gen:types:check` that diffs the live schema against the committed `lib/api-types.ts`. This will fire after L1 because the schema gains the `*_locale_actual` keys. Update the committed types in the same PR that lands S3.

---

11. L1 scope adjustments

Reading the spec against the codebase, here are the moves I recommend:

Add to L1

A1. Sitemap and robots.txt (extend S8 or split S8b). Already argued in §7. Half-day of work, naturally L1, lands ahead of any locale flip so search engines see the alternate-URL pattern from day one.

A2. `apiFetch` Accept-Language threading. Already argued in §9 R6. The frontend can't realistically meet S3's acceptance criteria without this — Django can't serve French content if the frontend doesn't ask for it. Currently implicit in S3; should be an explicit AC item.

A3. Composed middleware test coverage. §9 R1. Eight new test cases in `frontend/middleware.test.ts`, ~half a day. Without them the auth gate breaks silently under locale prefixes.

A4. Locale-aware caching test in S11. Add to the e2e suite: hit `/` with `Accept-Language: fr`, then immediately hit `/` with `Accept-Language: en`, assert each gets its expected locale. Catches Vercel cache-key misconfiguration. Half a day.

Slip from L1 to L3

B1. The `TranslationStatus` signal handlers. S4 spec says "Register in admin with read-only mode for L1." That's correct. The signal handlers I designed in §5 are L3 work — they only matter once content is being edited per-locale, which doesn't happen in L1 (catalogs are byte-identical English placeholders, and no `_fr` / `_de` / `_es` columns receive writes). **Confirm S4 explicitly excludes signal handlers** — current spec is ambiguous.

B2. `<field>_locale_actual` enforcement of `human_approved` gating. §4 mixin is L1; the gating logic that consults `TranslationStatus.status` to decide eligibility is L3. The L1 mixin should check column populated/empty only, not status. The `I18N_ENFORCE_REVIEW_GATE` setting flips True in L3.

Stay in L1 unchanged

- S1, S2, S5–S10, S11, S12 are correctly scoped.
- S3 and S4 stay with the small clarifications above.

Net L1 effort

Original estimate: ~7 working days. With A1–A4: +1–1.5 days. Net: ~8–8.5 days. Still inside the May 8 target with the May 1 start (5 working days, ~1 week slack).

---

12. Open questions surfaced (not in original D1–D18)

Two things came up in the audit that aren't addressed in D1–D18:

Q-NEW1 — `Taxon` translation tabs in admin

S2 registers `Taxon.common_family_name` as translatable. `Taxon` is currently surfaced in admin (it's an MPTT tree at `backend/species/models.py:79`). Modeltranslation's `TranslationAdmin` adds tabs per locale per registered field. Admin needs `TranslationAdmin` (or `TabbedTranslationAdmin`) inheritance; verify which admin class `Taxon` currently uses. Cosmetic L1 question — flag for the implementer.

Q-NEW2 — Search across translatable fields

`SpeciesViewSet.search_fields = ["scientific_name", "provisional_name", "common_names__name"]` (`backend/species/views.py:137`). None of these are translatable today. **No L1 search-localization work needed.** But: in L3, when French content lands, a French-speaking user searching "rouge" should match against `description_fr`. That's a deliberate L3+ decision (search localization affects rank stability and is a separate UX call). Document in `docs/planning/i18n/README.md` as an L3 open question. Flag for now; do not block L1.

---

13. Concerns (things to flag, not silently work around)

1. **The composed middleware pattern adds latency.** Every request to a protected route runs `next-intl` middleware *and* the auth gate. On Vercel's edge runtime this is fine (~1ms each), but it's visible in cold-start cost. Acceptable for L1, monitor in L4.

2. **`force-dynamic` pages bypass the locale-aware ISR cache entirely.** That's correct from a safety perspective but loses the perf win. The map page in particular is heavy and is now full-render-per-request × four locales. If this becomes a perf problem post-L1, the fix is to push the map's heavy data fetch into a route handler that *does* cache per locale, leaving the page render cheap.

3. **Modeltranslation registers fields globally per process.** A `translation.py` module loaded at app startup registers descriptors on the model class. There's no way to "register translation only for the species app" — once loaded, it's loaded everywhere. This is fine but worth knowing: tests that rely on the unsuffixed `description` to be a plain string column will silently start returning the active language's value. Affects any test that activates `fr` and reads `species.description`. **Add to S2 acceptance criteria: existing test suite passes against the post-S2 codebase.**

4. **The `*_locale_actual` keys will permanently exist.** Once shipped, removing them is a breaking API change. If we ever rethink the fallback-indication pattern (e.g., switch to a header-driven `Content-Language: en` to indicate fallback), we'd carry both. Acceptable trade — the keys are a small JSON cost — but it commits us to this shape.

5. **The conservation-writer agent extension (D8) is not under architecture review.** Per the prompt, the agent's contract is FYI-only. **However:** the agent's prompt presumably reads files under `docs/planning/i18n/glossaries/`, which don't exist yet. Make sure L3 (not L1) creates that directory and seeds it with empty per-locale files — otherwise the agent invocation in L3+ will fail on missing inputs.

6. **No fallback when DeepL is down.** D5 chooses DeepL but doesn't address availability. L3 will need a "DeepL request failed → mark TranslationStatus row as `mt_draft` with `notes` populated → retry job picks it up next pass" path. Out of scope for L1 but flag now so L3 doesn't discover it under deadline pressure.

7. **The `User.locale` field defaults to `en` for existing users.** That's correct but means every existing user keeps getting English emails post-L4 even if they prefer French. Self-service: the account page must surface the locale field in L4 alongside the rest of the profile editor. Flag for the L4 spec — currently L4's email work is described but the corresponding account-page UI isn't spelled out.

---

14. Hand-off to downstream agents

This architecture proposal does not modify `.claude/agents/*` beyond the conservation-writer extension already noted in the planning hub (which is FYI for this review).

The **PM agent writing the L1 spec revision** should pull the following acceptance-criteria additions from this doc:

- §1 — Locale-aware caching test in S11 (CI smoke test for cache key per locale).
- §2 — `frontend/lib/seo.ts` helper signature for S8.
- §3 — Permanent locale × auth × service-token tests for S1.
- §4 — `TranslationActualLocaleMixin` for S3 with `I18N_ENFORCE_REVIEW_GATE=False` in L1.
- §5 — `TranslationStatus` indexes and (for L3) signal handler design.
- §7 — Sitemap and robots.txt routes added to S8.
- §9 R1 — Composed middleware refactor and eight new test cases for `frontend/middleware.test.ts`.
- §9 R6 — `apiFetch` to thread `Accept-Language` from the active locale.
- §11 A1–A4 — net L1 scope adjustments (~1–1.5 day add).
- §11 B1–B2 — explicit L3 deferrals.

The **BA agent**, if invoked on L1, should treat:

- Section §10 (API contract) as the canonical answer to "is this breaking?" — not breaking, schema-additive.
- Section §13.4 as a regression-testing risk to call out.

The **test-writer agent** at the L1 verification gate should write tests for:

- The five locale × auth scenarios from §3.
- The eight middleware × locale scenarios from §9 R1.
- The two-back-to-back-requests cache-key test from §1 / §11 A4.
- The `*_locale_actual` keys returning `"en"` when no per-locale content exists, in DRF.

Files referenced

Absolute paths used in this review:

- `/Users/alekseisaunders/Repos/MadagascarFish/docs/planning/i18n/README.md`
- `/Users/alekseisaunders/Repos/MadagascarFish/docs/planning/i18n/gate-L1-framework.md`
- `/Users/alekseisaunders/Repos/MadagascarFish/docs/planning/architecture/registry-redesign-architecture.md`
- `/Users/alekseisaunders/Repos/MadagascarFish/CLAUDE.md`
- `/Users/alekseisaunders/Repos/MadagascarFish/frontend/middleware.ts`
- `/Users/alekseisaunders/Repos/MadagascarFish/frontend/middleware.test.ts`
- `/Users/alekseisaunders/Repos/MadagascarFish/frontend/lib/auth.ts`
- `/Users/alekseisaunders/Repos/MadagascarFish/frontend/lib/api.ts`
- `/Users/alekseisaunders/Repos/MadagascarFish/frontend/lib/coordinatorDashboard.ts`
- `/Users/alekseisaunders/Repos/MadagascarFish/frontend/lib/mapLocalities.ts`
- `/Users/alekseisaunders/Repos/MadagascarFish/frontend/app/layout.tsx`
- `/Users/alekseisaunders/Repos/MadagascarFish/frontend/app/page.tsx`
- `/Users/alekseisaunders/Repos/MadagascarFish/frontend/app/species/page.tsx`
- `/Users/alekseisaunders/Repos/MadagascarFish/frontend/app/species/[id]/page.tsx`
- `/Users/alekseisaunders/Repos/MadagascarFish/frontend/app/species/[id]/husbandry/page.tsx`
- `/Users/alekseisaunders/Repos/MadagascarFish/frontend/app/about/page.tsx`
- `/Users/alekseisaunders/Repos/MadagascarFish/frontend/app/dashboard/coordinator/page.tsx`
- `/Users/alekseisaunders/Repos/MadagascarFish/frontend/app/account/page.tsx`
- `/Users/alekseisaunders/Repos/MadagascarFish/frontend/components/SiteHeader.tsx`
- `/Users/alekseisaunders/Repos/MadagascarFish/frontend/package.json`
- `/Users/alekseisaunders/Repos/MadagascarFish/frontend/next.config.mjs`
- `/Users/alekseisaunders/Repos/MadagascarFish/backend/config/settings/base.py`
- `/Users/alekseisaunders/Repos/MadagascarFish/backend/config/urls.py`

- `/Users/alekseisaunders/Repos/MadagascarFish/backend/config/api_urls.py`
- `/Users/alekseisaunders/Repos/MadagascarFish/backend/accounts/models.py`
- `/Users/alekseisaunders/Repos/MadagascarFish/backend/accounts/views.py`
- `/Users/alekseisaunders/Repos/MadagascarFish/backend/accounts/urls.py`
- `/Users/alekseisaunders/Repos/MadagascarFish/backend/accounts/serializers.py`
- `/Users/alekseisaunders/Repos/MadagascarFish/backend/accounts/permissions.py`
- `/Users/alekseisaunders/Repos/MadagascarFish/backend/species/models.py`
- `/Users/alekseisaunders/Repos/MadagascarFish/backend/species/serializers.py`
- `/Users/alekseisaunders/Repos/MadagascarFish/backend/species/views.py`
- `/Users/alekseisaunders/Repos/MadagascarFish/backend/species/admin.py`